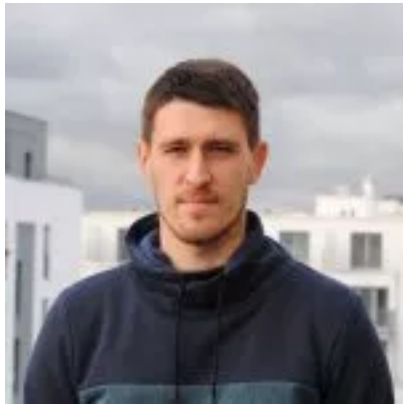


Language



Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



- suspend function
- scope
- builders
- exceptions
- flow

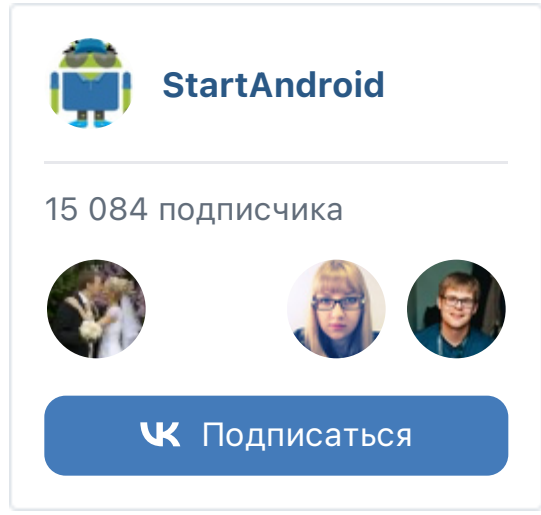
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

- Яндекс 410011180491924
- WebMoney R248743991365 Z551306702056
- Paypal

Урок 19. Hilt. EntryPoint

10 декабря 2021

В этом уроке разбираемся, как EntryPoint позволяет нам получить доступ к Hilt компонентам.

На прошлом уроке мы рассмотрели Hilt компоненты. Они создаются и инджелируются объекты подкапотно, и напрямую мы с ними не работаем. Но EntryPoint дает нам возможность взаимодействовать с компонентом так, как мы привыкли это делать в Dagger. Т.е. мы получим доступ к компоненту и сможем использовать его для получения объектов.

Давайте создадим простой EntryPoint, а потом разберемся, зачем нам может это пригодиться.

Описываем интерфейс с аннотацией @EntryPoint:

```
1 @EntryPoint
2 @InstallIn(ActivityComponent::class)
3 interface DatabaseEntryPoint {
4     fun getDatabaseHelper(): DatabaseHelper
5 }
```

С помощью @InstallIn мы указываем, какой именно компонент нас интересует - ActivityComponent.

Методы интерфейса описываются точно по тем же правилам, что и методы обычного Dagger компонента. Мы создаем get метод для получения DatabaseHelper объекта. Этот метод будет реализован в Activity компоненте в дополнение к его текущему функционалу.

Теперь нам надо получить этот компонент:

```
1 @AndroidEntryPoint
2 class MainActivity : AppCompatActivity() {
3
4     override fun onCreate(savedInstanceState: Bundle?) {
5         super.onCreate(savedInstanceState)
6         setContentView(R.layout.activity_main)
7
8         val entryPoint = EntryPoints.get(this, DatabaseEntryPoint::class.java)
9         val databaseHelper = entryPoint.getDatabaseHelper()
10    }
11 }
```

Это делается с помощью EntryPoints.get. На вход передаем Activity и наш EntryPoint интерфейс. Hilt использует Activity, чтобы получить его Activity компонент и вернет его нам, как реализацию нашего DatabaseEntryPoint интерфейса.

Далее мы вызываем get-метод, чтобы получить DatabaseHelper.

Тут важно понимать, что в переменной entryPoint у нас находится полноценный Activity компонент. Просто он скрыт от нас EntryPoint интерфейсом.

Чтобы убедиться в этом, можно сделать так:

```
1 (entryPoint as? App_HiltComponents.ActivityC)?.injectMainActivity(this)
```

Мы приводим entryPoint к классу Activity компонента. После этого мы можем вызвать, например, метод injectMainActivity, который обычно вызывается в super.onCreate. Особого смысла в этом нет, но это дает нам понять, что мы действительно работаем с Activity компонентом.

Таким образом с помощью EntryPoint мы получили Hilt компонент с нужным нам функционалом, описанным в EntryPoint интерфейсе.

Если нам нужен Application компонент, то в InstallIn передаем SingletonComponent::class, а в методе EntryPoints.get используем Application объект, а не Activity. Для остальных компонентов логика та же.

Зачем это может понадобиться? У меня есть два более-менее подходящих кейса.

Первый - это когда мы не знаем заранее какой объект мы будем использовать. Например, у нас в конкретном участке кода может понадобиться только один из трех объектов: FileHelper, DatabaseHelper, ServerApi. Не хотелось бы, что Hilt создавал все эти объекты, если нужен будет только один.

Можно, конечно, просто использовать Lazy. И когда мы точно будем знать, какой объект нам нужен, Hilt нам его создаст.

А можно создать EntryPoint, который научит компонент возвращать нам эти три объекта:

```
1 @EntryPoint
2 @InstallIn(ActivityComponent::class)
3 interface DataEntryPoint {
4     fun getFileHelper(): FileHelper
5     fun getDatabaseHelper(): DatabaseHelper
6     fun getServerApi(): ServerApi
7 }
```

Затем в коде мы просто вызовем нужный метод, а Hilt создаст объект и вернет его нам.

Второй кейс - это когда нам извне дают уже созданный объект. И нам доступны в нем только несколько override методов.

Хороший пример такого объекта - Activity. Оно создается системой и мы не можем в конструкторе прописать все, что нам нужно. Hilt умеет работать с Android объектами, но что если мы получим подобный объект извне? Hilt не сможет заинджелировать объекты в такой объект.

В качестве примера можно рассмотреть класс подключения:

```
1 class MyConnection: Connection {
2
3     override fun onConnect() {
4
5     }
6
7     override fun onDisconnect() {
8
9     }
10
11 }
```

Допустим, мы получаем готовую инстанцию этого класса из какой-то библиотеки. И в нем нам понадобится куча разных объектов. Можно конечно всех их создать где-то в нашем коде и потом просто передать сюда. Но если таких объектов много, то получится не очень удобно.

С помощью EntryPoint мы можем получить компонент, который умеет создавать все необходимые объекты. Далее мы просто передаем этот компонент в MyConnection, и там уже он предоставит нам все, что нужно.

А можно сделать еще проще. EntryPoint позволяет добавлять компоненту не только get-, но и Inject методы:

```
1 @EntryPoint
2 @InstallIn(ActivityComponent::class)
3 interface DataEntryPoint {
4     fun injectMyConnection(myConnection: MyConnection)
5 }
```

Нам остается прописать @Inject объекты внутри MyConnection:

```
1 class MyConnection: Connection {
2
3     @Inject
4     lateinit var databaseHelper: DatabaseHelper
5
6     override fun onConnect() {
7
8     }
9
10    override fun onDisconnect() {
11
12    }
13
14 }
```

И вызвать entryPoint.injectMyConnection, передав ему объект MyConnection. Компонент заинджелирует все необходимое.

Важное замечание! Hilt компонент, который мы получаем в entryPoint, может держать ссылку на соответствующую ему Android сущность. Т.е. ActivityComponent, например, может держать ссылку на Activity. Поэтому старайтесь не хранить этот компонент в тех объектах, которые живут дольше, чем Activity. Это может привести к утечке памяти.

НазадВперёд

Комментарии

Допустим, мы получаем готовую инстанцию этого класса из какой-то библиотеки. — Даниил

Вы сказали, что мы получаем готовый экземпляр MyConnection из библиотеки. Как тогда мы можем внутри этого класса добавить @Inject lateinit var databaseHelper: DatabaseHelper?

Ответить

RE: Допустим, мы получаем готовую инстанцию этого класса из какой-то библиотеки. — Dmitry Vinogradov

Примерно так же, как с Activity. Мы в нем можем прописать все что угодно, но при этом сами мы не создаем его экземпляры. Но должен признать, что кроме Android классов типа Activity и Service мне пока ни разу не встречались такие кейсы.

Ответить

Обновить список комментариев

RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

